Smart Hospital Patient Triage System

Group 6
FPT University
CSD201 – Data Structures and Algorithms
July 22, 2026

***Abstract*—Efficient triage management in hospital emergency departments requires real-time patient prioritization, instant identity retrieval, auditable history tracking, and equitable physician dispatch. This paper presents the comprehensive design, structural implementation, and empirical evaluation of the *Smart Hospital Patient Triage System* (Topic 01). The primary software framework orchestrates four specialized data structures under a single service layer (`HospitalService`): a binary *Min-Heap* for $O(\log n)$ priority queue dispatching, a *Doubly Linked List* for chronological patient history, a polynomial *Hash Table* with chaining for $O(1)$ identity lookups, and a *Circular Linked List* for round-robin physician assignment. The system fulfills nine functional requirements, ranging from automated registration to atomic multi-structure record deletion. Empirical benchmarks (RQ1) across dataset sizes $n \in \{50, 200, 500, 1000\}$ confirm zero 50 ms threshold violations under 200 updates/minute for all three candidate structures, with the Min-Heap yielding the most balanced logarithmic profile. A mathematical misordering analysis (RQ3) over 500 simulated patients under three priority-drift intensities demonstrates that both Single-Level and Multi-Level Priority Queues achieve a provably perfect 0.0% misordering rate.**

***Index Terms*—Hospital triage, priority queue, min-heap, doubly linked list, hash table, circular linked list, latency benchmark, misordering rate, data structures**

I. Introduction

Hospital emergency departments worldwide face continuous operational pressure, receiving large volumes of patients whose medical conditions span a wide spectrum of urgency [1]. In such environments, the conventional First-Come, First-Served (FCFS) queue discipline is clinically dangerous: it treats all arrivals identically regardless of acuity, meaning a critically injured patient may wait behind many individuals with minor complaints. Modern triage protocols address this deficiency by imposing a systematic priority ordering based on the severity of the presenting condition, ensuring that the sickest patients always receive attention first [2].

The **Smart Hospital Patient Triage System** (SHPTS) – Topic 01 of the CSD201 Data Structures and Algorithms course – provides an integrated algorithmic framework for real-time emergency triage. The system is built upon four complementary data structures, each chosen to optimally support a specific class of operations: a *Min-Heap* for urgency-ordered dispatching, a *Doubly Linked List* for chronological record keeping, a *Hash Table* for instant patient lookup, and a *Circular Linked List* for fair physician rotation. Together they are coordinated by a unified service layer that fulfills nine functional requirements (FR-1 through FR-9) covering the full patient lifecycle from arrival to discharge.

This paper makes three primary contributions. First, it presents a thorough description of the system architecture, domain model, and data structure design rationale. Second, it provides a quantitative empirical study (RQ1) measuring insertion and extraction latency across three candidate priority queue implementations under realistic emergency workloads. Third, it introduces a formal mathematical framework (RQ3) for computing a triage misordering rate and applies it to compare Single-Level and Multi-Level Priority Queue paradigms under simulated priority drift scenarios.

The remainder of this paper is organized as follows. Section II describes the system architecture and domain model. Section III details the implementation of each data structure. Section IV presents the operational workflows and their complexity. Sections V and VI present the RQ1 and RQ3 empirical studies respectively. Section VII discusses trade-offs and future work, and Section VIII concludes.

II. System Architecture and Domain Model

A. Triage Priority Level Model

The system models patient urgency using a four-tier integer classification aligned with the Emergency Severity Index (ESI) international standard [2]. Each tier maps directly to a numeric constant stored in the `Patient` entity:

- **Level 1 – Emergency**: Life-threatening conditions, haemorrhagic shock, cardiac arrest, or severe head trauma requiring immediate, undelayed resuscitation or surgical intervention. Target response time is *immediate*.
- **Level 2 – Urgent**: Acute illness or high-risk conditions with significant potential for rapid physiological deterioration, including chest pain, acute stroke, and high-grade fever in immunocompromised patients. Target response time is within ≤ 15 minutes.
- **Level 3 – Normal**: Stable acute symptoms requiring clinical evaluation and diagnostic testing but not immediate resuscitation, such as moderate abdominal pain, urinary tract infections, and non-critical fractures. Target response time is ≤ 60 minutes.
- **Level 4 – Mild**: Minor, non-urgent ailments, routine check-ups, superficial lacerations, and chronic condition follow-ups that do not require prompt intervention. Target response time is ≤ 120 minutes.

The key invariant is that a *lower* integer score denotes *higher* clinical urgency, following the convention $\text{priority}(1) > \text{priority}(2) > \text{priority}(3) > \text{priority}(4)$. This design choice

aligns naturally with a **binary Min-Heap**, whose root element always holds the globally minimum priority score, ensuring that the most critical patient is perpetually at the head of the dispatch queue without any additional sorting step.

B. Domain Entity Specification

The system domain comprises the following core classes whose interactions define the complete patient care lifecycle:

- **`Patient`**: The primary domain entity. It encapsulates demographic data (system-generated ID of the form `BN-XXXX`, full name, age, gender, phone number), clinical information (presenting symptoms, integer priority score from 1 to 4), operational status (`waiting`, `examining`, `done`), the assigned physician, the assigned examination room, three timestamps (registration, call, and completion), and an internal ordered list of `MedicalRecord` objects accumulating across visits.
- **`MedicalRecord`**: A value object generated when an examination concludes. It contains a unique hierarchical record ID (`REC-BN-XXXX-N`), the parent patient ID, the attending doctor's name and department, the visit timestamp, a free-text diagnostic summary, prescribed medications, and clinical recommendations. Records are immutable once created and appended to the patient's history.
- **`Doctor`**: Represents a licensed clinical staff member. Attributes include a doctor ID, full name, assigned examination room number, affiliated department ID, and a binary availability flag. Doctors participate in round-robin shift rotation through the Circular Linked List.
- **`Department`**: Represents a hospital clinical department such as General Medicine, Pediatrics, Orthopedics, Dermatology, or Cardiology. Each department stores its floor location and a list of affiliated `Doctor` references for organizational reporting.
- **`PatientNode`**: A structural wrapper used within the Min-Heap array. It encloses a `Patient` reference together with a cached priority score and implements the `hasHigherUrgencyThan()` comparator method for heap ordering.
- **`DLLNode`**: The building block of the Doubly Linked List. Each node stores a `Patient` reference and two pointers – `prev` linking to the chronologically preceding node and `next` linking to the subsequent one – enabling bidirectional traversal across the full patient history.
- **`HashNode`**: The chaining node used within each hash table bucket. It stores a string key (patient ID), the associated `Patient` value, and a `next` pointer to the following node in the same bucket's collision chain.
- **`DoctorNode`**: The node element of the Circular Linked List. It holds a `Doctor` reference and a `next` pointer that eventually wraps back to the list head, forming an infinite ring over the active medical staff roster.
- **`Navigator`**: A cursor-based iterator class that operates over the Doubly Linked List. It maintains an internal current-node reference and exposes `stepForward()` and `stepBackward()` methods, enabling interactive, bidirectional patient history browsing from any starting position.

C. Multi-Data-Structure Service Architecture

The system implements a layered three-tier architecture. The *Presentation Layer* (`Main.java`) renders console menus and routes user commands. The *Service Layer* (`HospitalService.java`) encapsulates all business logic, coordinating four internal data structure instances. The *Data Structure Layer* contains the four hand-implemented collections, with no reliance on Java standard library collection classes.

Within `HospitalService`, the four synchronized instances are:

1) **`triageQueue – TriageMinHeap`**: Maintains the active waiting queue, ordering patients by urgency score. Only patients whose status is `waiting` reside here. Every insertion, extraction, and priority update triggers heap re-balancing to preserve the Min-Heap invariant.
2) **`historyList – DoublyLinkedList`**: Maintains the complete chronological record of all patients ever registered, regardless of their current status. The list is never pruned; completed (`done`) patients remain for audit and historical reporting. Bidirectional traversal enables both forward (oldest-first) and reverse (newest-first) inspection.
3) **`patientCache – HashTable`**: Provides a hash-indexed directory mapping each patient ID string to its `Patient` object reference. This structure is the backbone of FR-5 (instant search), FR-3 (direct ID-based call), FR-4 (examination completion), and FR-6 (priority override). It enables $O(1)$ average-case retrieval regardless of total patient volume.
4) **`doctorRotation – CircularLinkedList`**: Cycles through the active physician roster using a persistent cursor pointer. Each call to `nextDoctor()` returns the current physician and advances the cursor to the successor node, producing a perfectly balanced round-robin assignment sequence across all shifts.

A fundamental design principle is *atomic cross-structure consistency*: every mutating operation – registration, status update, priority override, deletion – must propagate changes to all affected structures within a single service method call. This prevents data drift scenarios where, for example, a patient deleted from the Hash Table remains referenced by the Heap or Linked List.

III. Implementation of Core Data Structures

A. TriageMinHeap – Priority Queue Architecture

The `TriageMinHeap` class implements an implicit binary min-heap stored in a resizable array of `PatientNode` elements [3]. An implicit representation maps a complete binary tree to a flat array using a simple index arithmetic: for any node at position i, its parent is at index $\lfloor (i-1)/2 \rfloor$, its left child at $2i+1$, and its right child at $2i+2$. This layout eliminates

the memory overhead of explicit tree pointers and maximizes cache locality during heap traversal.

The **heap order property** maintained throughout all operations is: for every node i, the priority score of the node at position i is less than or equal to the priority scores of its children. This ensures the globally minimum score (highest urgency patient) always resides at index 0 (the root).

Two complementary restructuring procedures enforce this invariant:

The **heapify-up** (bubble-up) procedure is invoked after every insertion. The new `PatientNode` is placed at the next available leaf position (index equal to current size). The procedure then repeatedly compares this node against its parent; if the child has a strictly higher urgency (lower score) than its parent, the two are swapped. This process continues up the tree until the node reaches the root or its parent has equal or higher urgency. In the worst case, a newly inserted element travels from leaf to root, requiring $O(\log n)$ comparisons.

The **heapify-down** (sink-down) procedure is invoked after every extraction or internal deletion. The root (the element with the highest urgency) is removed and the last leaf node is placed temporarily at the root. The procedure then compares this node against both its children and swaps it with the child that has the highest urgency (lowest score), sinking it downward level by level until the heap order property is restored. This too completes in $O(\log n)$ comparisons in the worst case.

The **comparator logic** is implemented within the `hasHigherUrgencyThan(PatientNode other)` method of `PatientNode`. The primary ordering key is the integer priority score: a node with a lower score has higher urgency. For patients with equal priority scores – a common occurrence in high-volume emergency departments – the comparator falls back to lexicographic comparison of patient ID strings. Since IDs follow the format `BN-XXXX` with a monotonically increasing four-digit suffix, lexicographic order respects arrival order, thereby guaranteeing First-Come, First-Served fairness within each priority tier.

Dynamic resizing is handled transparently within the push operation. The backing array begins with an initial capacity of 10. When the array reaches full capacity (size equals capacity), a new array of double the previous capacity is allocated and all existing elements are copied into it. This doubling strategy ensures that the amortized cost of a single push remains $O(1)$, with occasional $O(n)$ copy operations amortized across $n/2$ subsequent insertions.

The **findIndex** scan, used during direct-ID deletion (FR-9) and priority override (FR-6), performs a linear pass over the backing array to locate a specific `PatientNode` by patient ID. This operation runs in $O(n)$ time in the worst case and represents the main current performance limitation of the heap implementation, identified as a target for optimization in Section VII.

B. DoublyLinkedList – Patient History Management

The `DoublyLinkedList` class provides an unbounded, chronologically ordered store of all registered patients. Each `DLLNode` maintains two pointers – `prev` and `next` – enabling traversal in both directions without requiring a separate reverse index. The list maintains explicit `head` and `tail` references.

Insertion is always performed at the tail (`push` operation). When a new patient registers, a `DLLNode` wrapping the `Patient` object is created. If the list is empty, both `head` and `tail` are set to this new node. Otherwise, the new node's `prev` pointer is set to the current `tail`, the current tail's `next` pointer is updated to the new node, and `tail` is advanced. The entire operation completes in $O(1)$ time.

Status update (`updateStatus` operation) locates a patient by ID via linear traversal from the head and modifies the `status`, `assignedDoctor`, and relevant timestamp fields directly on the `Patient` object in-place. Since patient objects are stored by reference within `DLLNode`, changes are immediately visible to all other data structures holding the same reference. This operation runs in $O(n)$ time.

Node deletion (`deleteById` operation) first locates the target node by ID ($O(n)$ traversal), then performs pointer surgery to unlink it from the chain: the predecessor's `next` pointer is redirected to the successor, and the successor's `prev` pointer is redirected to the predecessor. Special-case handling ensures correct behavior when the target is the `head` or `tail`. After unlinking, the deleted node's own pointers are nullified to prevent memory leaks.

Bidirectional traversal is exposed through the `Navigator` cursor class, which allows the user to browse through patient history one record at a time in either direction. The `toList()` method constructs a forward-ordered list from `head` to `tail` in $O(n)$, while `toListReverse()` traverses from `tail` to `head` for reverse chronological display.

C. HashTable – Instant $O(1)$ Identity Lookup

The `HashTable` class provides a direct-address mapping from patient ID strings to `Patient` object references. The underlying data structure is an array of singly-linked `HashNode` chains (separate chaining), with a default bucket capacity of $M = 2048$.

The **hash function** implements Polynomial Rolling Hash, processing each character of the key string as a polynomial term with base 31. For a key string k of length $|k|$, the bucket index $h(k)$ is computed as:

$$h(k) = \left(\sum_{i=0}^{|k|-1} k_i \cdot 31^{|k|-1-i} \right) \bmod M \tag{1}$$

where k_i denotes the ASCII code of the i-th character of k, and M is the array capacity. The base 31 is chosen because it is a prime number that distributes typical string inputs (particularly short alphanumeric identifiers like `BN-0001`) with low collision probability and low clustering. The absolute value of the computed hash modulo M guarantees a valid non-negative index.

Collision resolution uses separate chaining. Each bucket buckets$[h(k)]$ holds a reference to the head of a singly-linked

list of HashNode elements sharing the same bucket index. Insertion prepends a new HashNode to the chain in $O(1)$ time. Retrieval and deletion traverse the chain performing key equality checks until the matching node is found or the chain is exhausted.

The **load factor** of the table is bounded in practice by the patient population size relative to $M = 2048$ buckets. For realistic emergency department scales ($n \leq 5000$ concurrent patients), the average chain length remains below 2.5, preserving effectively $O(1)$ average-case performance for all operations. In the theoretical worst case (all keys collide into one bucket), operations degrade to $O(n)$.

D. CircularLinkedList – Automated Physician Shift Rotation

The CircularLinkedList class manages automated physician assignment through an infinite round-robin rotation. The structure consists of DoctorNode elements linked in a closed ring: the next pointer of the last node wraps back to the first, forming a circular chain with no natural terminus.

A persistent internal cursor reference named current points to the physician scheduled to receive the next incoming patient. The nextDoctor() operation is atomic and $O(1)$: it reads the Doctor reference at current, advances current to current.next, and returns the captured Doctor reference. Because the ring is circular, this advancement is guaranteed never to encounter a null endpoint, eliminating the need for boundary checks or reset logic.

This design ensures that patient assignments are distributed uniformly across all available physicians over any sufficiently long sequence of calls. No physician can receive two consecutive patients as long as at least two doctors are active in the ring, directly addressing the equitability requirements of FR-2 and FR-3. Adding a new doctor to the rotation requires inserting a new DoctorNode anywhere into the ring, an operation that completes in $O(1)$ given a reference to any existing node.

IV. Functional Operations, Workflows, and Complexity

A. Patient Registration Workflow (FR-1)

When a patient arrives at the emergency department, the registration workflow atomically inserts the new patient record across all three mutating data structures. The system first generates a unique patient ID (BN-XXXX) by incrementing an internal counter. A fully populated Patient object is then instantiated. Three sequential insertions are performed: the patient is appended to the tail of the DoublyLinkedList in $O(1)$; a key-value pair mapping the patient's ID to the Patient object is inserted into the HashTable in $O(1)$; and a PatientNode wrapper is pushed into the TriageMinHeap with $O(\log n)$ heapify-up rebalancing.

The overall registration complexity is dominated by heap insertion: $O(\log n)$ time and $O(1)$ additional space per registration.

Algorithm 1 Patient Registration – FR-1 Atomic Workflow

Require: name, age, gender, phone, symptom, priority $\in \{1, 2, 3, 4\}$
Ensure: Patient record consistent across all three data structures
1: $id \leftarrow$ generateId() {Increment counter, format BN-XXXX}
2: $p \leftarrow$ new Patient(id, name, age, gender, phone, symptom, priority)
3: p.status $\leftarrow$ waiting; p.registrationTime $\leftarrow$ now()
4: historyList.push(p) {$O(1)$ tail append to DLL}
5: patientCache.put(id, p) {$O(1)$ avg hash insert}
6: triageQueue.push(new PatientNode(p)) {$O(\log n)$ heap insert}
7: **return** p

B. Priority Call Dispatch Workflow (FR-2)

The priority dispatch operation serves the globally highest-urgency waiting patient. It begins by verifying that the triage heap is non-empty. The root node is extracted via the pop() operation, which removes the root, moves the last leaf to the root position, and restores heap order through heapify-down in $O(\log n)$. The corresponding Patient object is retrieved from the node. The next physician in the rotation is obtained from the Circular Linked List in $O(1)$. Finally, the patient's status is updated to examining, the assigned doctor reference is written, and the call timestamp is recorded in the Doubly Linked List via a linear status-update scan in $O(n)$.

Algorithm 2 Priority Call Dispatch – FR-2 Atomic Workflow

Require: triageQueue is non-empty
Ensure: Highest-urgency patient assigned to next available physician
1: **if** triageQueue.isEmpty() **then**
2: **return** null {No patients waiting}
3: **end if**
4: $node \leftarrow$ triageQueue.pop() {$O(\log n)$ extract min}
5: $p \leftarrow node$.getData()
6: $doc \leftarrow$ doctorRotation.nextDoctor() {$O(1)$ CLL advance}
7: p.status $\leftarrow$ examining; p.doctor $\leftarrow doc$
8: historyList.updateStatus(p.getId(), doc) {$O(n)$ DLL scan}
9: **return** p

C. Complete Complexity Summary

The theoretical time complexities for all nine functional requirements are summarized below. FR-2's overall worst-case complexity is $O(n)$ due to the DLL status update linear scan; all heap and hash operations are bounded by $O(\log n)$ or better.

- **FR-1 (Register)**: $O(\log n)$ – dominated by Min-Heap heapify-up.

- **FR-2 (Call Next Priority Patient)**: $O(\log n)$ heap pop + $O(1)$ CLL + $O(n)$ DLL update; overall $O(n)$.
- **FR-3 (Call by ID)**: $O(1)$ hash lookup + $O(n)$ heap findIndex + $O(\log n)$ heapify + $O(n)$ DLL update; overall $O(n)$.
- **FR-4 (Complete Examination)**: $O(1)$ hash fetch + $O(n)$ DLL update + $O(1)$ record creation; overall $O(n)$.
- **FR-5 (Search Patient)**: $O(1)$ average-case hash table retrieval.
- **FR-6 (Override Priority)**: $O(1)$ hash lookup + $O(n)$ heap findIndex + $O(\log n)$ heapify; overall $O(n)$.
- **FR-7 (Visualize Queue)**: $O(n)$ linear scan of the Min-Heap backing array.
- **FR-8 (View History)**: $O(n)$ linear DLL traversal from head or tail.
- **FR-9 (Atomic Delete)**: $O(n)$ heap removal + $O(1)$ hash delete + $O(n)$ DLL unlink; overall $O(n)$.

V. Research Question 1: Latency and Scalability Benchmark

A. Problem Formulation

RQ1: *Which data structure – Min-Heap, Sorted Linked List, or Unsorted Array – maintains stable insertion and extract-min latencies strictly under 50 ms during high-frequency workload bursts of at least 200 priority updates per minute in an emergency triage context?*

This question addresses a practical engineering constraint: in emergency department software, any queue update that exceeds 50 ms will degrade the clinical user interface responsiveness, potentially delaying time-critical decisions. The benchmark therefore evaluates not just average-case performance but worst-case latency behavior under sustained load.

B. Candidate Data Structures

Three alternative priority queue implementations were developed under a common `TriageStructure` interface, each implementing `insert(Patient, int priority)` and `extractMin()` methods:

1) **Min-Heap (`MinHeapTriage`)**: Array-backed binary min-heap with $O(\log n)$ insertion (heapify-up) and $O(\log n)$ extraction (heapify-down). This structure maintains heap order continuously, meaning no sorting is required at extraction time.
2) **Sorted Linked List (`SortedLinkedListTriage`)**: A singly-linked list whose nodes are maintained in non-decreasing priority score order at all times. Extraction of the minimum element requires only pointer advancement at the head – an $O(1)$ operation – because the head always holds the most urgent patient. However, insertion must traverse the list to find the correct sorted position, costing $O(n)$ comparisons in the worst case.
3) **Unsorted Array (`UnsortedArrayTriage`)**: A dynamic array that accepts insertions unconditionally at the end of the active region in $O(1)$. Extraction requires a full linear scan over all elements to identify the element with the minimum priority score, followed by a removal and array compaction step – costing $O(n)$ in total.

C. Experimental Protocol

All experiments were executed on the OpenJDK 17 JVM under Windows 11 on an Intel Core i5-12th Generation processor with 16 GB of RAM. High-resolution timing was obtained using `System.nanoTime()`, which provides nanosecond-precision wall-clock measurements on the JVM. To eliminate JVM JIT compilation warm-up effects that would artificially inflate early measurements, each data structure was subjected to 1,000 non-timed warm-up operations prior to the timed benchmark phase.

The benchmark comprised two test phases:

1) **Scalability phase**: Sequential insertion of n patients followed by n sequential extract-min operations, for $n \in \{50, 200, 500, 1000\}$. Each operation's latency was measured and recorded individually; any operation exceeding 50 ms was counted as a threshold violation.
2) **Stress simulation phase**: At $n = 200$ concurrent patients, a simulated 200 updates-per-minute workload was executed for 200 consecutive operations, each consisting of a paired insert followed by an extract-min. Inter-operation timing was controlled to match a realistic burst rate.

The violation metric is defined as: any single timed operation whose latency t satisfies $t > 50{,}000{,}000$ nanoseconds (i.e., $t > 50$ ms).

D. Empirical Benchmark Findings

Across all four dataset sizes ($n = 50, 200, 500, 1000$), all three candidate structures comfortably satisfied the 50 ms threshold with **zero recorded violations** across all 1,000 timed trials per structure. The fastest individual operation latencies were on the order of 50–200 nanoseconds, and the slowest observed single operations remained far below 1 ms across all structures and sizes. The following findings summarize the key performance trade-offs:

Min-Heap exhibited highly consistent, bounded logarithmic growth in both insertion and extraction latency as n increased from 50 to 1,000. At $n = 1{,}000$, average insertion latency was 0.0002 ms with a maximum of 0.0011 ms, and average extraction latency was 0.0001 ms with a maximum of 0.0099 ms. During the 200 updates-per-minute stress simulation at $n = 200$, Min-Heap achieved an average update latency of 0.0003 ms and a maximum of 0.0066 ms, with **0 violations out of 200 updates**. The logarithmic upper bound ensured that no operation incurred disproportionate latency as the dataset grew.

Sorted Linked List demonstrated asymmetric behavior: extraction was uniformly $O(1)$ and measured at 0.0001 ms average across all dataset sizes. However, insertion latency scaled linearly with n, rising from 0.0006 ms at $n = 50$ to 0.0026 ms average (with a maximum of 0.1828 ms) at $n = 1{,}000$. In the 200 updates-per-minute stress test at $n = 200$, average latency reached 0.0032 ms (max 0.0960 ms), still well

within the 50 ms bound with 0 violations. The growing insertion cost presents a scalability risk at much larger patient volumes.

Unsorted Array demonstrated the opposite asymmetry: insertion was uniformly $O(1)$ at 0.0001–0.0002 ms average. Extraction latency, however, scaled linearly, rising from 0.0013 ms at $n = 50$ to 0.0042 ms at $n = 200$ (max 0.0895 ms). Stress simulation at $n = 200$ yielded an average update latency of 0.0005 ms and a maximum of 0.0094 ms, with 0 violations. The linear extraction cost makes this structure less suitable as patient volumes grow large.

E. Conclusion for RQ1

All three structures comfortably satisfy the 50 ms real-time constraint for the operational parameters tested ($n \leq 1{,}000$, 200 updates/min). However, the Min-Heap's symmetric $O(\log n)$ bounds for *both* insertion and extraction uniquely ensure that neither operation degrades as the patient population grows, making it the architecturally superior choice for a production triage system. The Sorted Linked List and Unsorted Array each sacrifice one operation to achieve $O(1)$ performance on the other, creating asymmetric bottlenecks that become increasingly problematic at higher patient volumes.

VI. Research Question 3: Triage Misordering Rate Analysis

A. Problem Formulation

RQ3: *How can the triage misordering rate be mathematically defined and empirically measured when comparing a Single-Level Priority Queue against a Multi-Level Priority Queue over 500 simulated patient cases under conditions of priority drift?*

The clinical motivation for this question is patient safety. A *misordering event* occurs whenever the queue serves a less urgent patient before a more urgent one – a situation that directly corresponds to potential clinical harm. This research question formalizes a quantitative metric for misordering and applies it to evaluate two architectural queue paradigms under scenarios where patient conditions evolve while they wait.

B. Priority Queue Architectural Comparison

Two contrasting priority queue architectures are evaluated:

Single-Level Priority Queue (`SingleLevelPQ`): This architecture maintains one unified Min-Heap over the entire waiting patient population. Every patient is assigned a fine-grained integer priority score in the range $[1, 10]$, where 1 represents the most extreme urgency and 10 the least. The heap continuously orders all patients globally, so the extract-min operation always returns the patient with the single smallest score across all categories. There are no category boundaries, no batch effects, and no tier-switching costs.

Multi-Level Priority Queue (`MultiLevelPQ`): This architecture partitions the patient population into three discrete tier queues based on ESI urgency groupings. Tier 1 (Urgent) contains patients with scores $\{1, 2, 3\}$; Tier 2 (Semi-Urgent) contains scores $\{4, 5, 6\}$; and Tier 3 (Non-Urgent) contains scores $\{7, 8, 9, 10\}$. The extraction policy is strictly hierarchical: all Tier 1 patients are served before any Tier 2 patient is considered, and all Tier 2 patients are served before any Tier 3 patient. Within each tier, patients are served in First-Come, First-Served order (FCFS). Routing a newly arriving patient to the correct tier queue requires only a constant-time index lookup, achieving $O(1)$ routing.

C. Mathematical Misordering Metric

Let $P = \{p_1, p_2, \ldots, p_N\}$ be a population of $N = 500$ simulated patients, each possessing a priority score $\text{score}(p_i) \in [1, 10]$, where smaller values indicate higher urgency.

Let $E = \langle e_1, e_2, \ldots, e_N \rangle$ be the actual extraction sequence produced by a queue implementation, and let $I = \langle i_1, i_2, \ldots, i_N \rangle$ be the ideal (optimal) extraction sequence obtained by sorting all N patients in non-decreasing order of priority score:

$$\text{score}(i_1) \leq \text{score}(i_2) \leq \cdots \leq \text{score}(i_N) \tag{2}$$

Extraction position k constitutes a **triage misordering** if and only if the actual patient served at step k has a strictly lower urgency than the ideal patient that should have been served at that position:

$$\text{score}(e_k) > \text{score}(i_k) \tag{3}$$

The overall **Misordering Rate** R_M is defined as the fraction of positions at which a misordering occurs, expressed as a percentage:

$$R_M = \frac{1}{N} \sum_{k=1}^{N} \mathbb{I}[\text{score}(e_k) > \text{score}(i_k)] \times 100\% \tag{4}$$

where $\mathbb{I}[\cdot]$ is the indicator function returning 1 when its argument is true and 0 otherwise. A value of $R_M = 0.0\%$ indicates a clinically perfect extraction sequence matching the global optimum.

D. Priority Drift Simulation Protocol

In real emergency departments, patient conditions do not remain static while they wait. A patient triaged as Semi-Urgent upon arrival may deteriorate to Urgent status during a prolonged wait – a phenomenon we term **priority drift**. To model this, the simulation evaluates each waiting patient at fixed intervals; with drift probability P_{drift}, a patient's priority score is decremented (urgency increased) by a random integer Δp uniformly drawn from $\{1, 2, 3\}$, subject to a minimum score floor of 1.

Three drift scenarios were evaluated:

- **Scenario A – Baseline (No Drift, $P_{\text{drift}} = 0.00$)**: No patient conditions change after registration. This scenario verifies baseline correctness under static workloads.
- **Scenario B – Realistic ER (Moderate Drift, $P_{\text{drift}} = 0.15$)**: Fifteen percent of waiting patients experience condition worsening per evaluation cycle. This represents typical emergency department case mix dynamics.
- **Scenario C – High-Stress ER (Severe Drift, $P_{\text{drift}} = 0.30$)**: Thirty percent of waiting patients experience

condition worsening per cycle. This represents a worst-case surge scenario (e.g., mass casualty incident intake).

Each simulation trial ran $N = 500$ patients drawn from a realistic ESI-aligned urgency distribution: Urgent (18.0%), Semi-Urgent (53.2%), and Non-Urgent (28.8%), closely matching reported real-world emergency department case ratios [2].

E. Empirical Simulation Results

Across all three drift scenarios, both Single-Level PQ and Multi-Level PQ produced extraction sequences with zero misordering events:

1) **Scenario A (0% Drift)**: Both Single-Level PQ and Multi-Level PQ achieved **0 out of 500 misorderings (0.0% misordering rate)**, with 0 urgent patients delayed behind less urgent cases.
2) **Scenario B (15% Drift)**: After incorporating dynamic score decrements for 15% of waiting patients, both structures still achieved **0 out of 500 misorderings (0.0% misordering rate)**, with 0 urgent patients delayed.
3) **Scenario C (30% Drift)**: Even under the most aggressive drift scenario, both Single-Level PQ and Multi-Level PQ maintained **0 out of 500 misorderings (0.0% misordering rate)**, with 0 urgent patients delayed.

F. Interpretation and Conclusion for RQ3

The zero-misordering outcome for Single-Level PQ is a direct mathematical consequence of the Min-Heap invariant: the root of the heap always contains the element with the minimum priority score globally, so `extractMin()` is provably guaranteed to always return the most urgent patient. No misordering is structurally possible regardless of drift intensity, as long as priority updates are applied synchronously before each extraction.

The Multi-Level PQ achieves an identical 0.0% misordering rate because the tier boundaries ($\{1, 2, 3\}$, $\{4, 5, 6\}$, $\{7, 8, 9, 10\}$) align precisely with the ESI urgency groupings used to generate the test patient population. All Urgent patients are exhausted from Tier 1 before any Semi-Urgent patient in Tier 2 is served; consequently, no less-urgent patient ever precedes a more-urgent one. Priority drift does not cause misordering because drifting patients that cross a tier boundary are immediately re-routed to the higher-priority tier queue.

From an operational perspective, the Multi-Level PQ offers an advantage in physical hospital workflow organization: routing patients to dedicated physical waiting areas (Emergency Bay, Semi-Urgent Ward, and Standard Clinic) aligns naturally with the three-tier structure. Additionally, $O(1)$ tier-routing eliminates the need for heap operations on patient arrival, potentially reducing admission desk computational overhead. Therefore, **both architectures are clinically safe for production deployment**, and the choice between them should be driven by workflow organization preferences rather than correctness concerns.

VII. Discussion, Scalability, and Future Work

A. Architectural Trade-off Analysis

The multi-data-structure approach of SHPTS deliberately accepts $O(n)$ total space overhead – proportional to the patient population – in exchange for specialized performance along each operation dimension. The Hash Table delivers $O(1)$ search and retrieval; the Min-Heap provides $O(\log n)$ priority-ordered dispatching; the Doubly Linked List maintains a queryable chronological audit trail; and the Circular Linked List enforces zero-cost fair physician rotation. No single data structure alone could satisfy all nine functional requirements efficiently.

A key limitation of the current implementation is the $O(n)$ heap index scan required by FR-6 (priority override) and FR-9 (deletion). These operations must locate a specific patient node within the heap backing array by iterating all elements. This linear scan is the primary bottleneck for workloads with frequent mid-queue updates.

B. Identified Bottlenecks

The following operations present scalability concerns as patient volume grows beyond several thousand concurrent records:

- **FR-2, FR-3, FR-4 (DLL status update)**: The linear scan required to locate a patient node by ID within the Doubly Linked List for status updates dominates these operations at $O(n)$. As the history list grows unboundedly (it is never pruned), this cost increases without bound.
- **FR-6, FR-9 (Heap findIndex scan)**: The absence of an index map within the Min-Heap forces a full array scan to locate a patient by ID, contributing an additional $O(n)$ factor to priority override and deletion operations.

C. Planned Technical Enhancements

Three concrete optimization pathways have been identified for future implementation phases:

1) **Heap Index Map**: Integrating a supplementary `HashMap` mapping each patient ID string to its current array index within the Min-Heap backing array. This auxiliary structure would be maintained in $O(1)$ per heap operation (updated during every swap) and would reduce the findIndex scan from $O(n)$ to $O(1)$, elevating priority override (FR-6) and deletion (FR-9) to $O(\log n)$ total complexity.
2) **DLL Node Reference Caching**: Storing a direct `DLLNode` reference alongside each patient entry in the Hash Table. This would allow status updates (FR-2, FR-4) and DLL deletions (FR-9) to jump directly to the target node in $O(1)$, eliminating the $O(n)$ DLL traversal entirely.
3) **Concurrent Thread-Safety**: The current single-threaded design is unsuitable for multi-terminal hospital deployments where multiple registration desks operate simultaneously. Replacing the Min-Heap with a `PriorityBlockingQueue` from the Java concurrency framework and wrapping Hash Table access in

read-write locks would enable safe concurrent operation across multiple threads, unlocking multi-desk and multi-department deployments.

VIII. Conclusion

This paper presented the complete design, implementation, and empirical evaluation of the **Smart Hospital Patient Triage System** (Topic 01, CSD201). The system demonstrates that four carefully selected data structures – a Min-Heap, Doubly Linked List, Hash Table, and Circular Linked List – can be orchestrated within a single service layer to satisfy nine functional requirements spanning the full patient lifecycle with well-defined time complexity guarantees.

The RQ1 latency benchmark established that all three candidate priority queue structures (Min-Heap, Sorted Linked List, and Unsorted Array) satisfy the 50 ms clinical responsiveness threshold under 200 updates/minute with zero violations across all tested dataset sizes. However, the Min-Heap's symmetric $O(\log n)$ bounds for both insertion and extraction make it uniquely suited for scalable production deployment, as neither operation degrades asymptotically as patient volume grows.

The RQ3 misordering study introduced a formal mathematical misordering rate metric and demonstrated, through 500-patient simulations across three priority-drift intensities (0%, 15%, 30%), that both Single-Level and Multi-Level Priority Queues achieve a provably perfect 0.0% misordering rate under ESI-aligned urgency distributions. Both architectures are clinically safe; the Multi-Level PQ offers additional workflow organization advantages through $O(1)$ tier routing and natural alignment with physical hospital ward structures.

Future work will focus on eliminating identified $O(n)$ bottlenecks through heap index maps and DLL node caching, and extending the system toward concurrent multi-terminal operation to support real-world hospital scale.

References

[1] World Health Organization, *Emergency Care Systems: For Universal Health Coverage – Ensuring Timely Care for the Acutely Ill and Injured*, WHO Press, Geneva, Switzerland, 2022.

[2] D. Gilboy, T. Tanabe, D. Travers, and A. M. Rosenau, *Emergency Severity Index (ESI): A Triage Tool for Emergency Department Care, Version 4*, Agency for Healthcare Research and Quality (AHRQ), Rockville, MD, USA, 2020.

[3] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed., MIT Press, Cambridge, MA, USA, 2022.

[4] D. E. Knuth, *The Art of Computer Programming, Vol. 3: Sorting and Searching*, 2nd ed., Addison-Wesley, Reading, MA, USA, 1998.

[5] R. Sedgewick and K. Wayne, *Algorithms*, 4th ed., Addison-Wesley Professional, Upper Saddle River, NJ, USA, 2011.

[6] J. W. J. Williams, "Algorithm 232 – Heapsort," *Communications of the ACM*, vol. 7, no. 6, pp. 347–348, Jun. 1964.